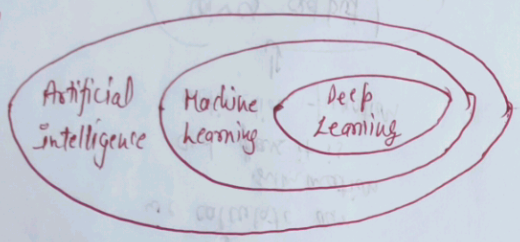
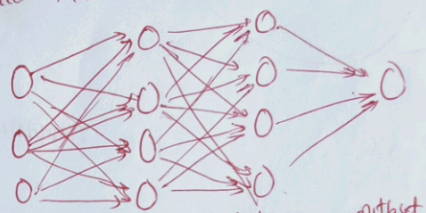
Deep Learning

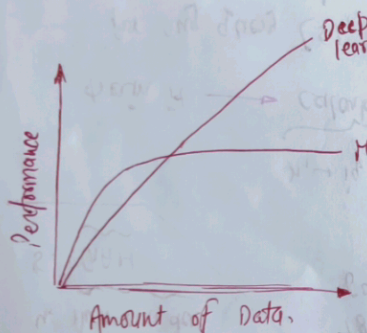
based on artificial neural networks with representation learning

feature learning



↳ attempts to draw similar conclusion as humans would by continuously analyzing data with a given logical structure called Neural Network.





some architecture :-

- image classification : RESNET
- Text classification : BERT
- Image segment : UNet
- image translation : Pix2Pix
- object Detection : YOLO
- Speech Detection : WaveNET

low layer → edges
higher layer → human relevant concepts
} image processing

Why Now?

- Dataset
- Hardware
- Frameworks
- Architecture
- Community

1) Data Dependency → High data Hungry
2) Hardware Dependency → need GPU
3) Training Time → High (1 month/week)
4) Feature Selection → automatic
5) Interpretibility

Types of Neural Network :-

1. Multi-layer Perceptrons
2. CNN
3. RNN
4. Auto encoders
5. GAN

training:

iq	cgpa	placed
78	7.8	1
69	5.1	0

x_1 → iq x_2 → cgpa.

train it → calculate w_1, w_2, b

for any query (51, 3.9)

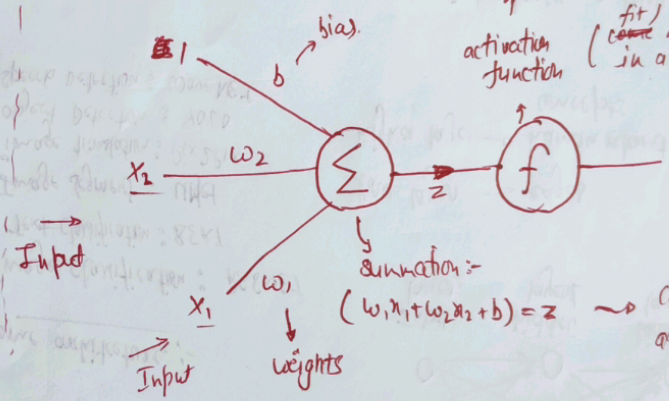
we calculate our summation and pass it in activation function

gives output / prediction

What is Perceptron?

- it is an algorithm used for supervised machine learning
- It is building block of deep learning

design :-



- mathematical function/model

eg :- step func", sigmoid func", etc.

activation function (fit / ~~come~~ make z in a range)

→ can classify data which are sort of linear.

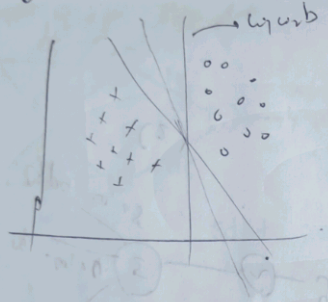
difference b/w neurons & perceptrons :

1) Complex
2) processing is difference
3) Neuroplasticity

Perceptron Trick ?

github - nden

Loss function :- quantify how good is result

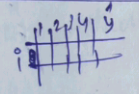


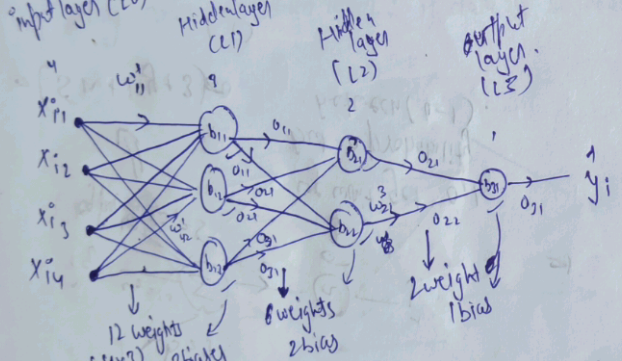
loss → $f(w, w_2 b)$ → Rational no.

Loss function $(L) = \frac{1}{n} \sum_{i=1}^{n} \max(0, -y_i f(x_i))$

where, $f(x_i) = w_1 x_{i1} + w_2 x_{i2} + b$

MLP Notation :-





total : (15) + (8) + (3) → 26 - trainable parameter.

Data → (m × n)

m → # rows

n → 4 (~~x_1, x_2, x_3, x_4~~)

each row denoted by x_i

$(w\,b)$ → notation

w, b, o

b_{ij} (bias) i → layer, j → node no.

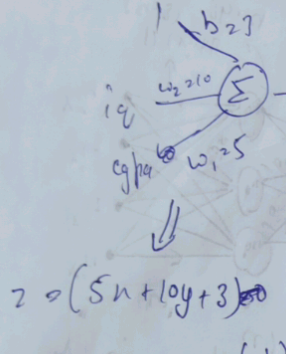
o_{ij} (output) i → layer, j → node no.

w^k_{ij} i → current layer mai kon se node se nikal rha.

j → kaha pe ghus rha hai

k → kon si ...

Multi Layer Perceptron :- assumption :- loss func^n → log loss , activation func^n → sigmoid , model → logistic regression (classification)

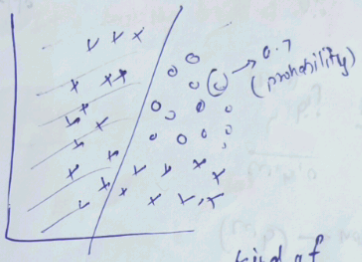


$z = (5x + 10y + 3)$

(1)

we won't get 0,1 but a probability between (0-1).

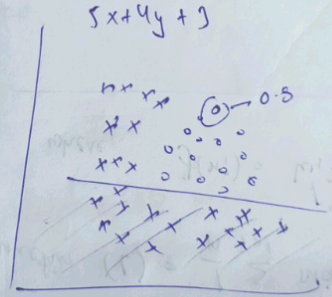
$5x+10y+3$

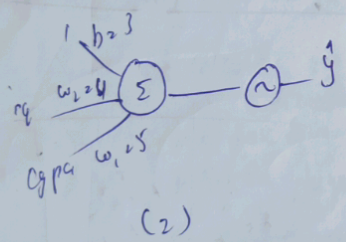


kind of [superposition] = ~~smoothening~~

+

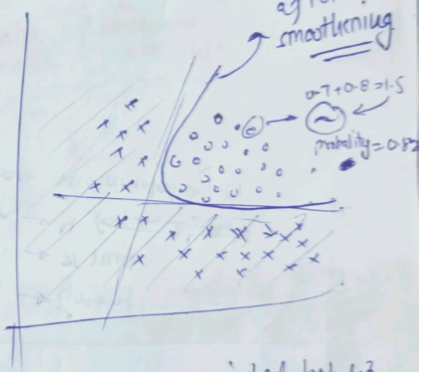
$5x+4y+3$





(2)

this curve after smoothening



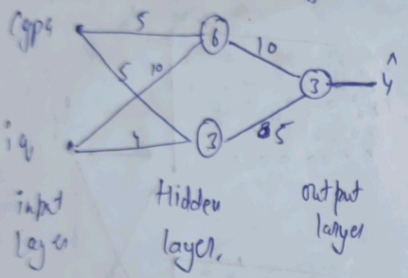
or can use weighted probability and bias (b) = 3.

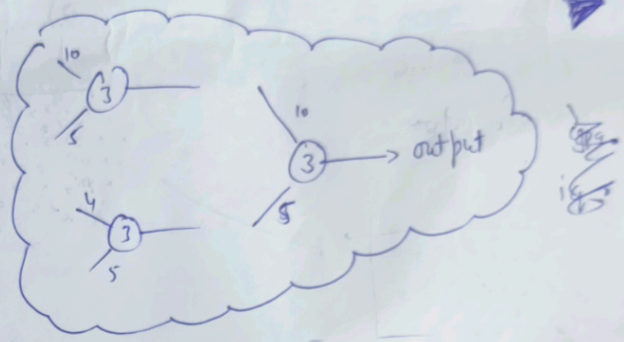
$(b + 0.7 \times 10 + 0.8 \times 5) \Rightarrow \sigma \Rightarrow 0.81$

weights can be predict using accuracy of again an model

this whole stuff is nothing but a perceptron

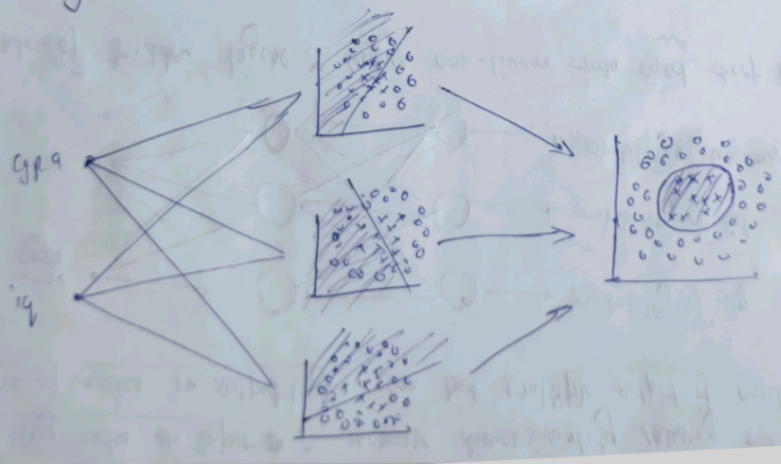
Multilayer Perceptron





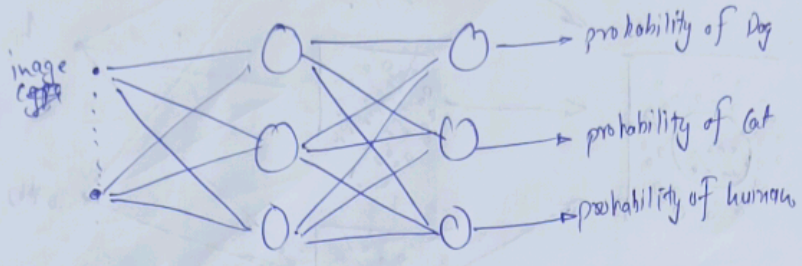
Making changes to neural network :- (study of neural network architecture

1) Adding nodes in hidden layer :- if data is so much non-linear, so can capture that by increasing no. of nodes



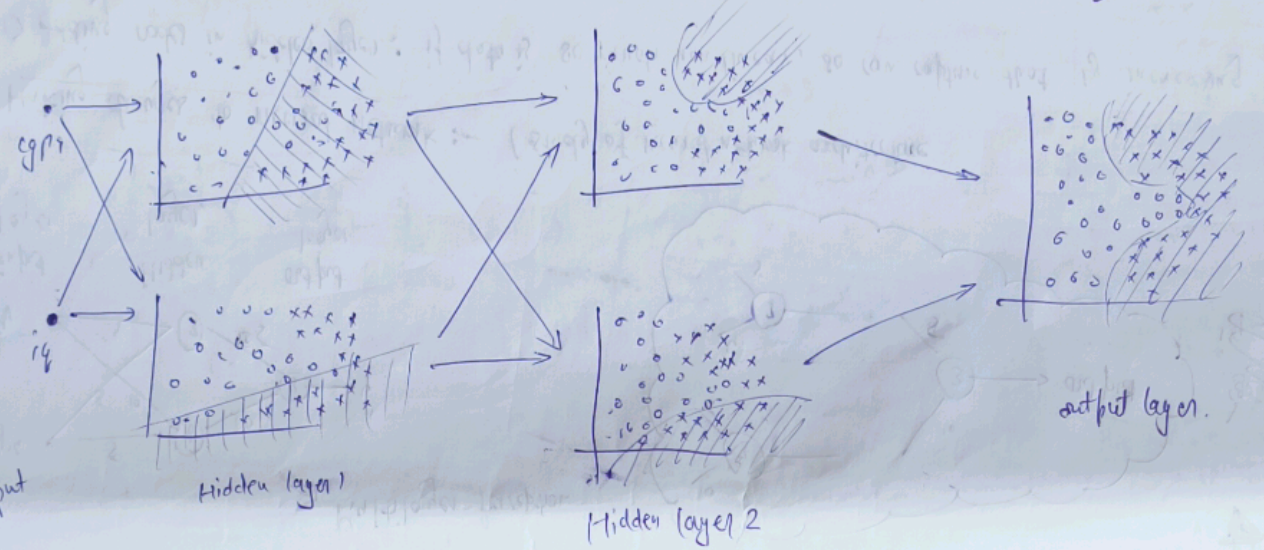
2) Adding nodes in input : increase dimensionality causing more work

3) No. of nodes in output layer: Can put multiple output if multiclass classification



Note:- that's why they are called (UFA) universal function approximator as it can capture any pattern after enough time and train.

4) No. of hidden layers: Complex non-linear data need deep layer



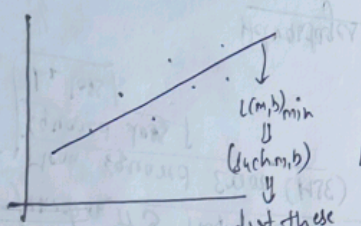
What is a Loss function:-

⇒ Loss function is a method of evaluating how well your algorithm is modelling in your dataset.

value → high ≈ poor
↳ low ≈ great

$f(x) = x+2$

L(parameter) ⇓ of model.



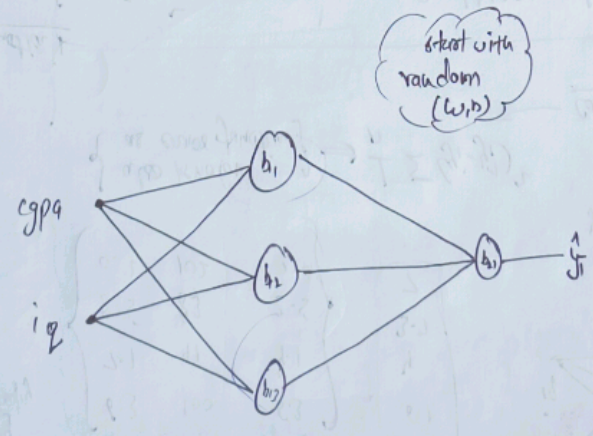
$L = (y_i - \hat{y_i})^2$ ⇒ mean square error.

$mx_i + b$.

$L(m,b) = (y_i - mx_i - b)^2$

start with random (w,b)
calculate L using forward propagation
⇓
using gradient descent, adjust values of (w,b)
⇓
do untill some epochs so that to get as close as to Lmin.

cgpa	iq	package (lpa)
7.1	83	3.2
8.5	91	4.5
6.3	102	6.1
5.1	87	2.7
⋮	⋮	⋮



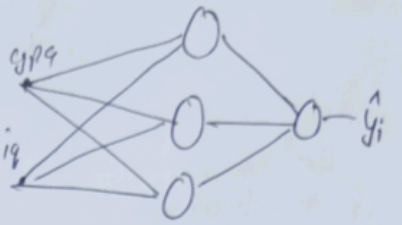
forward propagation

Loss functions in DL :-

- Regression
  - mean_squared_error
  - mean_absolute_error
  - huber loss
- Classification
  - binary cross entropy
  - categorical cross entropy
  - hinge loss
- Auto encoders
  - KL divergence
- GAN
  - discriminator loss
  - min max
  - gan loss.
- object detection
  - focal loss
- embedding
  - ~~triplet~~ loss

Loss function vs cost function :-

cgpa	iq	package	prediction
6.3	100	6.3	6.1
7.1	91	4.1	4
8.5	83	8.5	3.7
9.2	102	7.2	7



Loss function → single training eg

$y_i = 6.3 \quad \hat{y}_i = 6.1$

$(y_i - \hat{y}_i)^2 = (6.3-6.1)^2 = 0.04$.

{ also known as error function } $\Rightarrow \frac{1}{n}\sum (y_i - \hat{y}_i)^2$ ← Cost function → complete batch.

~~loss function~~ #5 important loss function :-

1. Mean Squared Error (MSE) :- $L = (y_i - \hat{y}_i)^2$ or (true - predict)^2

Squared loss / L_2 loss

$C = \frac{1}{n}\sum (y_i - \hat{y}_i)^2$

Why squaring ?
↓
negative value
↓
take for doing overall error

Advantages :-
1) Easy to interpret
2) differential
3) 1 local minima.

disadvantage :-
1) unit of error (square).
2) not Robust to outliers (x).
3) work only on linear model.

2. Mean absolute error (MAE) :- $L = |y_i - \hat{y}_i|$

L_1 loss

$C = \frac{1}{n}\sum |y_i - \hat{y}_i|$

Advantage :-
1) intuitive
2) same unit
3) robust to outliers

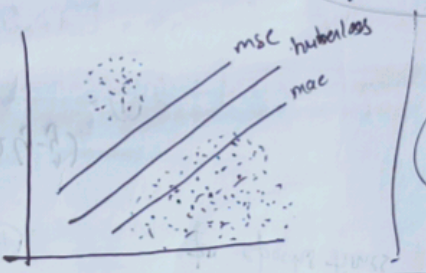
Disadvantage :-
1) not differentiable (have to use subgradients)

3. Huber loss :-

$$L = \begin{cases} \frac{1}{2}(y-\hat{y})^2 & \text{for } |y-\hat{y}| \le \delta \\ \delta|y-\hat{y}| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

↓ performs well when there are a lot of outliers

otherwise → mae
δ → hyperparameter.



~~used~~ used in softmax regr.

4. Binary cross entropy :- (or log loss)

→ classification
→ Two classes (0,1)

$L = -y\log(\hat{y}) - (1-y)\log(1-\hat{y})$

y → actual value
$\hat{y}$ → prediction

Advantage : + differentiable
disadvantage : × not intuitive
× multi local minima

$$L = -\sum_{j=1}^{k} y_j \log(\hat{y}_j)$$

(where k is # class in the data.

$$C = -\sum_{i=1}^{n}\sum_{j=1}^{k} y_{ij}\log(\hat{y}_{ij})$$

5. Categorical cross entropy

cgpa	iq	placed
8	80	Yes
6	60	No
7	70	maybe

Yes	No	Maybe
1	0	0
0	1	0
0	0	1

made by using one hot encoding

y_1 y_2 y_3 ... y_k (k=3)

activation function
↓
softMax

$z_1 \to$ yes
$z_2 \to$ No
$z_3 \to$ Maybe

$f(z_i) = \frac{e^{z_i}}{e^{z_1}+e^{z_2}+e^{z_3}}$

$0 \le z_1, z_2, z_3 \le 1$
$z_1+z_2+z_3 = 1.$

$L = -y_1\log(\hat{y}_1) - y_2\log(\hat{y}_2) - y_3\log(\hat{y}_3)$

(suppose) 0 (0.1) 1 (0.6) 0 0

$L = -\log(0.6)$.

(sparse categorical cross entropy)
↓ faster.

every thing is same, just use one hot encoding differently

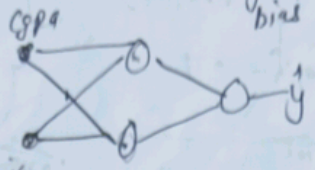
cgpa	iq	placed	OHE
—	—	yes	1
—	—	No	2
—	—	Maybe	3

during loss have to neglect correctly as there 0 doing work.

Back-Propogation :- algorithm to train neural network.

weights / bias ⇒ Most correct value

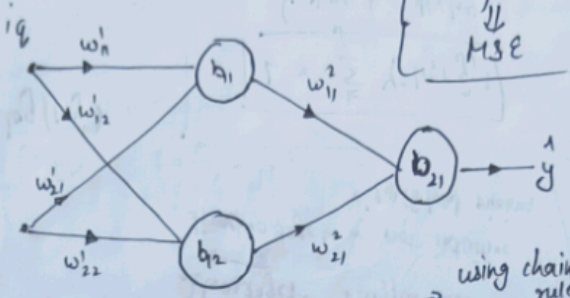
cgpa	iq	lpa
8	80	8
7	70	7



activation fun ⇓ linear

loss function ⇓ MSE

iq	cgpa	lpa
80	8	3
60	9	5
70	5	8
120	7	11



$\hat{y} \rightarrow o_{21}$

$o_{21} = w^2_{11} o_{11} + w^2_{21} o_{12} + b_{21}$

$o_{11} = w^1_{11}(iq) + w^1_{21}(cgpa) + b_{11}$

$o_{12} = w^1_{12}(iq) + w^1_{22}(cgpa) + b_{12}$

using chain rule ⇒

$$\frac{\partial L}{\partial w^2_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^2_{11}} \quad -(\#)$$

$$\frac{\partial L}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}} (y - \hat{y})^2 = -2(y - \hat{y}) \quad -(1)$$

step 1 : a) you select a point (row) ↳ student

a) init $(w, b) \rightarrow (1, 0)$

2) predict (lpa) → foward prop (dot product) day (18 lpa) instead of

3) Choose a loss function : MSE → $(3-18)^2 = 225$ (error)

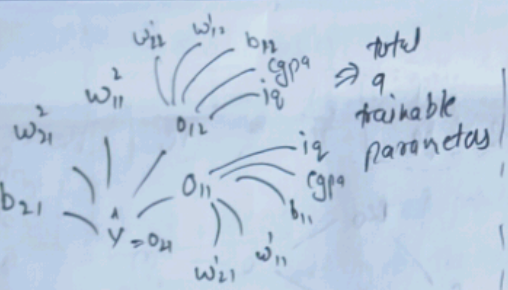
4) by back mecarcy we get L (parameter) , weight and bias → update.

gradient descent

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial w_{old}}$$

$$B_{new} = B_{old} - \eta \frac{\partial L}{\partial B_{old}}$$

5) do this in loop step 2-4 for epochs times



⇒ total 9 trainable parameters

⇓

the heavary which we have to change goes backward that is why we call it back propagation

$$\frac{\partial \hat{y}}{\partial w^2_{11}} = \frac{\partial}{\partial w^2_{11}} [o_{11} w^2_{11} + o_{12} w^2_{21} + b_{21}] = o_{11} \quad -(11)$$

using (1) & (11) in (#)

$$\Rightarrow \frac{\partial L}{\partial w^2_{11}} = -2(y - \hat{y}) o_{11}$$

this is the way to calculate $\frac{\partial L}{\partial w_i}$

very intuitive : similarly $\frac{\partial L}{\partial w^1_{21}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial o_{11}} \frac{\partial o_{11}}{\partial w^1_{21}}$

$$\frac{\partial L}{\partial w^k_{ji}} = -2(y - \hat{y}) o_{(k-1)j}$$

$$\frac{\partial L}{\partial b_{ij}} = -2(y - \hat{y})$$

Why does it work?

↳ Concept of gradient decent in higher dimension.

What is convergence? → ≈0. with time

$$w_n = w_0 - \eta \frac{\partial L}{\partial w}$$

while → convergence $\left(\eta \frac{\partial L}{\partial w} \approx 0\right)$ minimum condition for L.

⇓

however, it is written as :

iterate for 100 epochs

Using Memoization in Backpropagation

$\left(\hat{y} = w^3_{11} o_{21} + w^3_{21} o_{22} + b_{23}\right)$

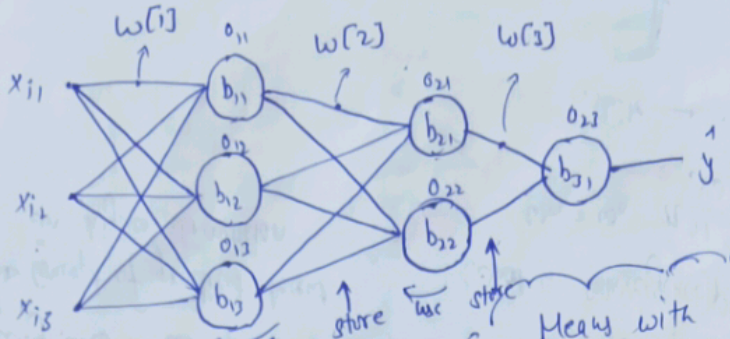
$L = f(\hat{y})$.

Total derivative law

Note!:

$h(f(x), g(x))$ ← $f(x) \leftarrow x$, $g(x) \leftarrow x$

$$\frac{\partial h}{\partial x} = \frac{\partial h}{\partial f} \times \frac{\partial f}{\partial x} + \frac{\partial h}{\partial g} \times \frac{\partial g}{\partial x}$$



Means with increase in no. of hidden layer the difficulty of doing derivative increases exponentially. Thus we would most likely to use memoization

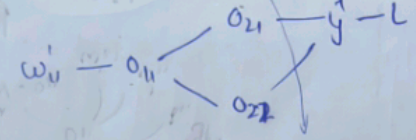
Backpropagation = chain rule (maths) + memoization (comp.)

easily all two layer

$$\frac{\partial L}{\partial w^3_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial w^3_{11}}$$

$$\frac{\partial L}{\partial w^2_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial w^2_{11}}$$

difficult for two layer ex: w'_{11}



$$\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial \hat{y}} \left[\frac{\partial \hat{y}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w'_{11}} + \frac{\partial \hat{y}}{\partial o_{22}} \times \frac{\partial o_{22}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w'_{11}} \right]$$

Types of Gradient Descent

There are three types of gradient descent, which differ in how much data we use to compute the gradient of the objective function

1. Batch GD
2. Stochastic GD
3. Mini GD

or

$$\therefore \frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial \hat{y}} \left[\frac{\partial \hat{y}}{\partial o_{21}} \times \frac{\partial o_{21}}{\partial o_{11}} + \frac{\partial \hat{y}}{\partial o_{22}} \times \frac{\partial o_{22}}{\partial o_{11}} \right] \frac{\partial o_{11}}{\partial w'_{11}}$$

Gradient Descent :- most popular algorithm to perform optimization and by far the most common way to optimize neural network.

(already studied in ML)

How Backpropagation use gradient descent:

```
epochs = 5
for i in range(epochs):
    for j in range(x.shape[0])
        → select one row (random)
        → predict (using forward propogation)
        → Calculate loss (using → mse or other)
        → update weights and bias using GD
            ∴ [ w_n = w_o - η ∂L/∂w ]
    → calculate avg loss for the epochs
```

Batch GD (Vanilla GD) :-

```
for i in range(nb_epochs):
    param_grad = evalute_gradient(loss_fun, data, params)
    params = params - learning_rate * param_grad
```

(minus)

↳ entire data

↳ if data have 50 points/row

so in 1 epoch:

↳ ~~50 times~~ (randomly choosing) predict and ~~update(w,b)~~

predict all 50 rows in one ~~iteration~~ go with updating → $\hat{y}$ → array(50)

↓

find loss of all y's (or you say cost).

$$\sum_{i=0}^{50} (y - \hat{y})^2$$

↓

one single time update in each epoch.

Which is faster? (Given same no. of epochs)

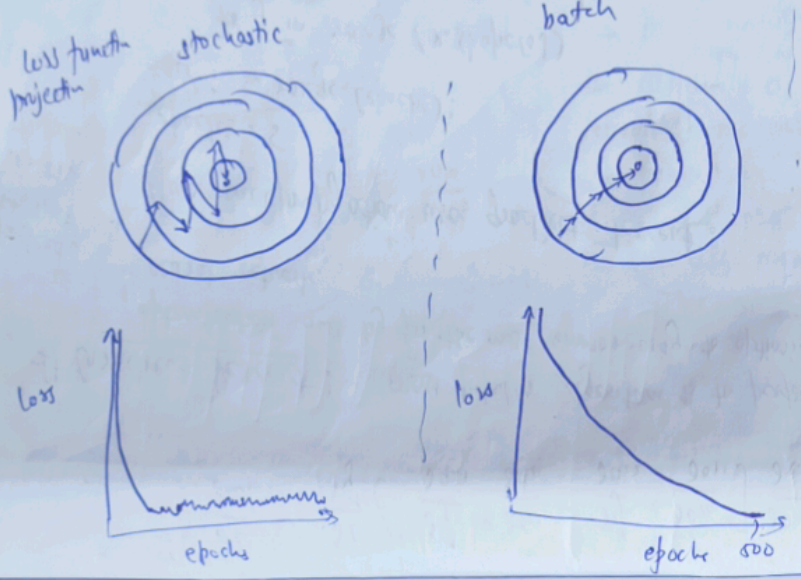
↳ batch > stochastic

: as no. of updates are more in stochastic.

Which is faster to converge? (same no. of epochs)

↳ batch < stochastic

as no. of updates are more in stochastic.



Stochastic GD

```
for i in range (nb_epochs):
    np.random.shuffle(data)
    for example in data:
        params_grad = evaluate_gradient(loss_func, example, param)
        param = param - learning_rate * params_grad.
```

↳ 1 random point
↳ y-hat and loss
↳ wb update

so if 10 epochs and 50 rows, no. of updates would be = 50 × 10 = 500

~~Thus it is~~

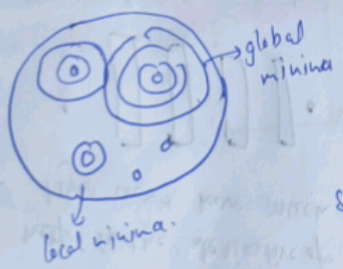
during code do:

model.fit(X, y, epochs=10, batch_size=320)

if 320 → batch GD.
if 1 → stochastic GD.

---

spikyness in SGD useful?



BGD → will stuck in local minimum in some cases, depending upon its starting position

SGD → can come out of local minimum due to the random jumps

~~disadv~~ disadvantage: BGD is more accurate than SGD

SGD don't converge exactly near, it is also near but less than BGD.

Mini Batch Gradient Descent

no. of rows → 320.

a trade of kind between batch and stochastic

in every epochs
10 batches
↓
of 32 each.

```
for i in epochs
    for j in no. of batches
        i'batch
        ↳ y_pred
        ↳ loss
        ↳ update.
```

no. of updates = no. of batch × no. of epochs.

Problem with vectorization (np.dot()),

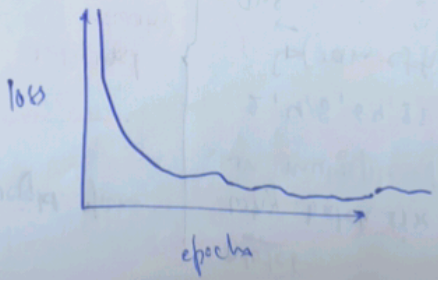
↓
if the no of rows are too many then it would be difficult to load that in ram.

Advantage of vectorization
↓
faster as it is optimized
↳ than looping

mbgd can also handle problem of vectorization, as we load the data in batches on ram.

:→ speed: bgd > mbgd > sgd

:→ convergence: bgd < mbgd < sgd.

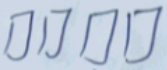


~~# Vanashi~~

Vanishing Gradient Problems:-

↳ The gradient will be vanishingly small, effectively preventing the weight from changing its value.

1) 0.1 x 0.1 x 0.1 x 0.1 = 0.0001 → VGD

2) Deep NN → 

3) Sigmoid / tanh ← activation function.

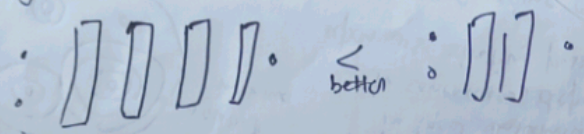
eg:-

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial z} \times \frac{\partial z}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial w_{11}}$$

How to handle?

① Reduce Model complexity:

help so the derivative of shallow layer won't have much terms



disadvantage: we increase layer to find ~~feat~~/train complex pattern, so reducing them will backfire the main goal.

Note:-

Why batch_size is provided in multiple of (2)?

2, 4, 8, 64, 32, 128, 256

↳ (ram effective usage)

ans.

↳ prevent memory padding

How to improve a neural network?

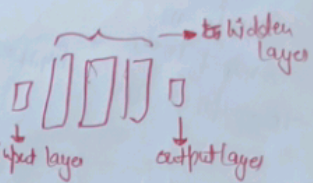
1. Fine tuning NN hyperparameters

- no. of hidden layer
- neurons per layer
- learning rate in GD
- Optimizer
- Batch_size
- Activation func
- Epochs

2. By solving problems:

- Vanishing / Exploding gradient
- Not enough data
- Slow training
- ~~Out~~ Overfitting

① No. of hidden layers:



By experience:-

1 hidden layer with 512 neurons (better)

3 hidden layer with 32 neurons

↳ Why?

the starting layer will go for primitive features and as we go further (face recognization) the layers recognize the complex pattern.

eg:-

starting layers → lines, curves (simple).

middle layers → shapes, complex curves

end layers → objects, final designs

↳ representative learning.

How much? → until overfitting, DNN will work good.

② Using ReLU activation function:-

max (0, z)

derivative: 0 or 1.

③ Proper Weight init

④ Batch norm → layer

⑤ Residual Network ← CNN → ResNET

↳ building block → ANN

② No of neurons:- → in input and output layers the no. of neurons depend on dataset.

↳ with experience:- try to start with sufficient nodes and keep reducing if you face overfitting.

③ Batch-Size:-

- Batch
- stochastic
- Mini Batch

two views

smaller (8 to 32)

- → training slow
- → generalization better

large (8192)

- → training fast
- → if generalize better that it is also fast

no depends upon GPU ram.

By experience:

try to start with smaller learning rate and increase it with increasing epochs.

↓↓

warming up the learning rate

④ Epochs → any → 100, 500, 1000 & keras jyada se jyada

↳ concept of early stopping

↓

keras check if there is no change than it stops there.

↳ implement by Call backs.

5) Vanishing and exploding gradients :-
- weight init
- Activation function
- Batch normalization
- Gradient clipping

6) No enough data :-
- for transfer learning
- unsupervised pretraining

7) Slow training :-
- optimizers
- learning rate schedules

8) Overfitting :-
- l_1 and l_2 regularization
- dropouts

Early stopping :-

callbacks = [
monitor = "val_loss", → to be monitor
min_delta = 0, → minimum change in monitor value
patience = 0, → no of epoches
verbose = 0,
mode = "auto",
baseline = None,
restore_best_weights = False,
]

Data Scaling :-

important as the model will take time or won't converge.

Standardization	Normalization
$\frac{x_i - x_{mean}}{\sigma}$	$\frac{x_i - x_{min}}{x_{max} - x_{min}}$
-1 to +1	0 to 1.

Dropout layers in DL :-

→ first of all we have to know about overfitting

→ It is said "ANN are prone to overfitting", due to their complex structure.

→ ① add more data
② Reduce complexity by reducing layer
③ Early stopping
④ Regularization → L_1, L_2
⑤ Dropout

Note:- dropout only during training, not during testing.

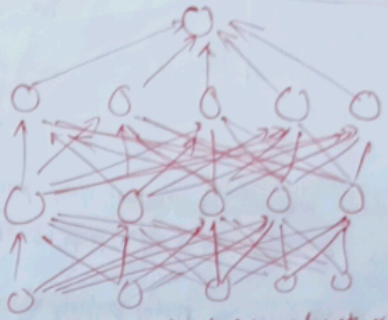
for epochs 1 → randomly remove some portion of nodes
for epochs 2 → randomly remove some nodes
so on.

means in each epoch you are training the data to different neural network

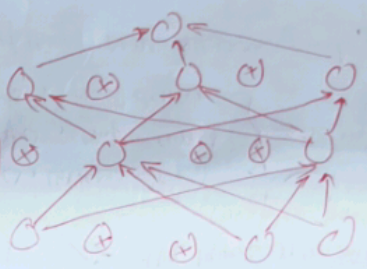
p = 0.5 → portion of nodes

after training each node will change with the weight to :-

$w = w(1-p)$ → probability to stay.



(a) standard Neural network



(b) After dropout

To solve overfitting → reduce no. of nodes of neural network
~~→ so that won't capture very small possibilities~~
~~→ reduce make the model~~
→ train the model, so that it won't go for small pattern but see the overall big picture

good strategy
→ start with last layer
drop → 40-50%
drop → 10-50%

→ Solve by dropout

How?

say the first node of hidden layer 5 depends too much on node no. 3 of of layer 4. it will value all the nodes of layer 4 as we sometime drop that node 3 of layer 4 in some epochs.

eg:- if 50% employs are chosen randomly to some how will it benefit the company?

second eg:- random forest analogy (make different trees on random data and then combine it).

in dropout also we kind of making different trees ~~and then~~ and training them simultaneously instead of how we do in random forest one at a time).

(ensemble learning.)

$0.2 < p < 0.5$ → good result

Regularization :- → so we found our weights/bias by minimizing loss function

or for a batch we name this loss/error as cost funct

$$C = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{y}_i)$$

in regularization we add a penality term :-

(Hyperparameter)

$$C = \left[\frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{y}_i)\right] + \text{penality term}$$

$\omega \neq 0$.

α regularization

$$\frac{\lambda}{2n}\sum_{i=1}^{k} \|w_i\|^2 \quad (L_2)$$

$\lambda \uparrow$ underfitting
$\lambda \downarrow 0$ overfitting

$$\frac{\lambda}{2n}\sum_{i=1}^{k} \|w_i\| \quad (L_1) \quad \omega \neq 0$$

final loss/cost function :-

$$C = \frac{1}{n}\sum_{i=1}^{n} L(y_i, \hat{y}_i) + \sum_{l=1}^{l}\sum_{i=1}^{}\sum_{j=1} \|w_{ij}^{l}\|^2$$

intuition behind Regularization :-

$$L' = L + \frac{\lambda}{2n}\|w_i\|^2$$

$$\frac{\partial L'}{\partial w_0} = \frac{\partial L}{\partial w_0} + \frac{\lambda}{2n} 2w_0$$

$$\frac{\partial L'}{\partial w_0} = \frac{\partial L}{\partial w_0} + \frac{\lambda w_0}{n}$$

$$w_n = w_0 - \eta\left(\frac{\partial L}{\partial w_0} + \frac{\lambda w_0}{n}\right)$$

$$w_n = w_0 - \eta\frac{\partial L}{\partial w_0} - \eta\frac{\lambda w_0}{n}$$

$$w_n = \left(1 - \frac{\eta\lambda}{n}\right)w_0 - \eta\frac{\partial L}{\partial w_0}$$

this is also called weight decay.

$\left(1 - \frac{\eta\lambda}{n}\right) w_0 < w_0$.

thus $w_n \approx 0$ eventually

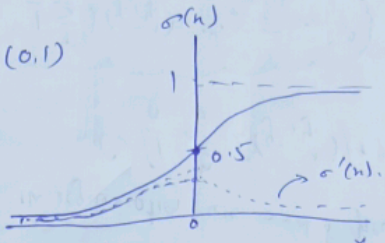
Activation Functions :- is a kind of mathematical gate for the input and output of a neuron.

Ideal Case →
- Non-linear
- Differentiable (to use GD)
- Computationally inexpensive
- Zero centered (normalized) eg- tanh
- Non-Saturating (must not have a range). eg:- Relu

(Saturating → vanishing gradient)

① Sigmoid function :- (0,1)

$$\sigma(n) = \frac{1}{1+e^{-n}}$$



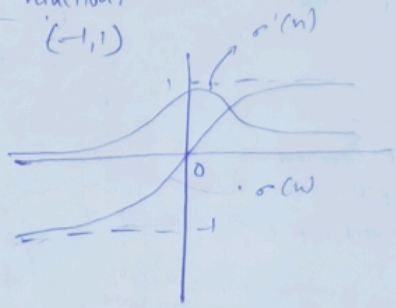
Doraemon gadget

Advantage :-
1. in binary classification it works to calculate probability of any event
2. Non-linear function
3. differentiable. → Backpropogation

~~Dist~~ Disadvantage :-
1. Saturating function → [0,1] (→ vanishing Gradient).
2. Non-zero centered. (all gradient → +ve or -ve)
3. Computationally expensive.

② tanh activation function :- (-1,1)

$$\sigma(n) = \frac{(e^{x} - e^{-x})}{e^{x} + e^{-x}}$$

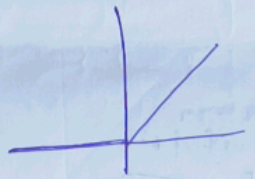


Advantage :-
1. Non-linear
2. Differentiable
3. zero centered. → } training faster

Disadvantage :-
1. Saturating function. → vanishing gradient
2. Computationally expensive.

③ Relu function :-

$$f(n) = \max(0, x)$$



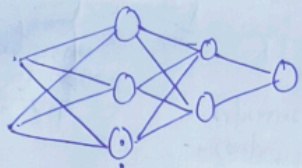
Advantage :-
1. Non-linear
2. ~~Not~~ Not saturated
3. Computationally inexpensive
4. Convergence → faster

Disadvantage :-
1. Not-differentiable. (assumption ($n<0$ $\sigma'(n)=0$; $n\geq 0$ $\sigma'(n)=1$)
2. Non zero centered
   ↳ can solved by batch normalization

Solution :-
- → set low learning rate
- → bias → +ve value
- → Don't use relu, instead use variant

~~Finish~~

Dying Relu Problem :-



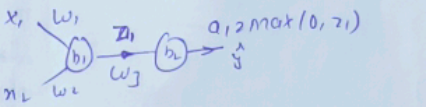
dead neuron.

for any given input, output is zero.

if more than 00% ~~data~~ neuron become dead node.

⇩

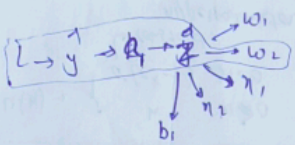
model can't capture the pattern of complex.



$z_1 = w_1 x_1 + w_2 x_2 + b_1 < 0$.

$a_1 = \max(0, z_1) = 0$

$\frac{\partial a_1}{\partial z_1} = 0$ → weights won't be updated



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial a_1} \times \left(\frac{\partial a_1}{\partial z_1}\right) \times \frac{\partial z_1}{\partial w_1}$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \times \frac{\partial \hat{y}}{\partial a_1} \times \left(\frac{\partial a_1}{\partial z_1}\right) \times \frac{\partial z_1}{\partial w_1}$$

if $z > 0$

$\frac{\partial L}{\partial w_1} > 0$.

now if η is great/large.

$w_n = w_0 - \eta \frac{\partial L}{\partial w_1}$

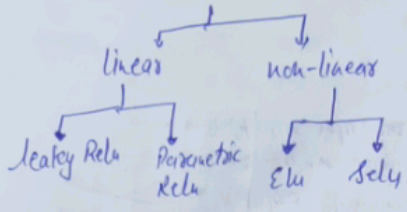
$w_n < 0$

dying relu. ← $z < 0$

Why it happen

↳ high negative bias.

Variant of Relu:-



① Leaky Relu

$f(z) = \max(0.01z, z)$

$z \geq 0 \rightarrow z$

$z < 0 \rightarrow \frac{1}{100} z$

Advantage:-
1. Non-Saturated
2. easily Computed.
3. No dying Relu problem
4. Close to zero center

Disadvantage:-
1. why 0.01?

② Parametric Relu :-

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ ax & \text{otherwise} \end{cases}$$

a → trainable parameter → depend on data.

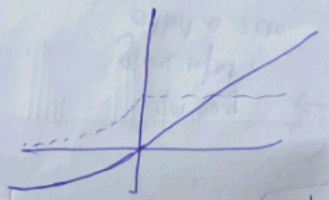
extra

Advantage:-
1. flexible than Leaky relu.

③ Elu - Exponential linear Unit :-

$$ELU(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

↳ constant, hyperparameter.



→ perform better than Relu
↳ may be because it is continuous and differentiable.

Advantage:-
- → Close zero centered ↳ convergence faster
- → better generalized
- → Dying relu ✗
- → Always continuous as well differentiable.

Disadvantage
- → Computationally expensive

④ Selu :- Scaled Elu.

$$SELU'(x) = \lambda \begin{cases} 1 & x > 0 \\ \alpha e^x & x \leq 0 \end{cases}$$

↓ to normalize converges faster.

α, λ → fixed hyperparameter

Why weight initializing is important?

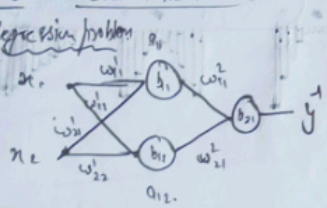
- vanishing gradient
- exploding gradient
- slow convergence

} Problems.

what weights not to do :-

CASE 1 → zero initialization

Regression problem



① relu :- $a_{11} = \max(0, z_{11}) \rightarrow 0$

$z_{11} = \omega^1_{11} x_1 + \omega^1_{21} x_2 + b_{11} = 0$

$a_{12} = \max(0, z_{12}) \rightarrow 0$

$z_{12} = \omega^1_{12} x_1 + \omega^1_{22} x_2 + b_{12} = 0$

during back propagation:-

$\omega^1_{11} = \omega^1_{11} - \eta \frac{\partial L}{\partial \omega^1_{11}}$ → 0 (calculate)

$(\omega^1_{11})_{(n)} = (\omega^1_{11})_{(o)}$ ⇒ no update

last ⇓

no training (100% dead relu).

② tanh :-

$a_{11} = \frac{e^{z_{11}} - e^{-z_{11}}}{e^{z_{11}} + e^{-z_{11}}}$

$a_{11} = \frac{1-1}{1+1} = 0$

⇓

$\frac{\partial L}{\partial \omega^1_{11}} = 0$

$(\omega^1_{11})_{(n)} = (\omega^1_{11})_{(o)}$

⇓

no update (100% dead relu)

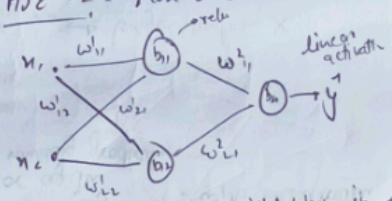
③ Sigmoid :-

$z_{11} = 0$.

$a_{11} = \text{sigmoid}(z_{11}) = \text{sig}(0)$

$a_{11} = 0.5$

CASE → 2. Non-0 constant value.

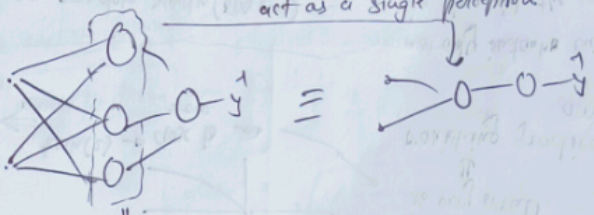


(say) $\omega = 0.5$ $b = 0.5$

$z_{11} = z_{12}$. } → same as sigmoid of zero.
$a_{11} = a_{12}$ ⇓ linear model.

This will happen in all subcases → relu, tanh, sigmoid

⇓ act as a single perceptron.



⇓

as they are trained and changes equally causing no variations

Because Single node or linear model

also $a_{11} = a_{12}$
$z_{11} = z_{12}$

⇓

$\frac{\partial L}{\partial \omega^1_{11}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{11}} \frac{\partial a_{11}}{\partial z_{11}} \frac{\partial z_{11}}{\partial \omega^1_{11}}$ (x_1)

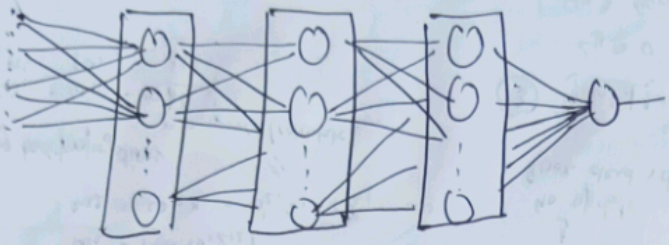
$\frac{\partial L}{\partial \omega^1_{12}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{12}} \frac{\partial a_{12}}{\partial z_{12}} \frac{\partial z_{12}}{\partial \omega^1_{12}}$ (x_2)

$\frac{\partial L}{\partial \omega^1_{21}} = \frac{\partial L}{\partial \omega^1_{12}}$ → Probabilistic

$\frac{\partial L}{\partial \omega^1_{21}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_{11}} \frac{\partial a_{11}}{\partial z_{11}} \frac{\partial z_{11}}{\partial \omega^1_{21}}$ (x_2)

$\frac{\partial L}{\partial \omega^1_{22}} = \frac{\partial L}{\partial \omega^1_{21}}$ (always)

CASE-3. → Random Initialization : ~→ small
~→ large



say $x_1, x_2, \ldots, x_{1000}$ / y — distributing gaussian

weights init → random
↓
small values

np.random.randn(500, 500) x 0.01

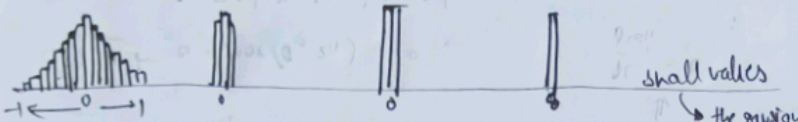
bias = 0

$x_i \rightarrow (0-1)$ scaled.

$z = \sum w_i x_i$

$w_i \rightarrow$ small.

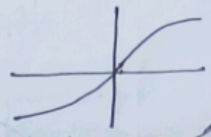
z → small no.



small values

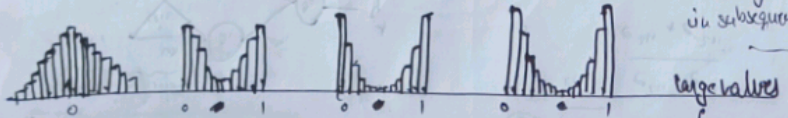
↳ the gaussian distribution tends to converge to zero in subsequent layer.

for tanh



tanh(z) → close to zero. ⇐ recall

This causes the gradient calculation is very small.
⇓
vanishing gradient arise.



large values

↳ slow convergence or again vanishing gradient.

in case of relu, which is unsaturated, the gradient will be unstable. as very higher jumps will be seen.

np.random.randn(500, 500)
↳ $w \rightarrow (0-1)$ (large)

$\sum w_i x_i \rightarrow$ saturation.

not only in tanh but in sigmoid the value of $a_{ij} = 0.5$ initially which causes vanishing gradient if network is deep

→ zeros , → non-zero x , → small random x , → large random x

Then how to initialize?

↳ Heuristics (jugad) → practical soln

- xavier / Glorot init → normal / uniform (tanh, sigmoid)
- He init → normal / uniform (relu).

} Keras implements it.

~~Initialization~~ ~~random~~ (small)
~~Xavier~~ np.random.randn(250, 450) x 0.01
(large)
np.random.randn(2, 4, 250) x 1

1) Xavier → normal init ⇒ multiply by $\sqrt{\frac{1}{fan\_in}}$

⇓
no. of inputs coming from previous layer.

or variance = $1/n$

np.random.randn(250, 250) * $\sqrt{\frac{1}{250}}$
↳ standard deviation

in case of relu.
⇓ ↳ derived by complex maths and experiment

2) He normal → factor of $\sqrt{\frac{2}{fan\_in}}$

for uniform distribution

3) Xavier uniform → [-limit, limit]
↳ random no. choose

$\therefore limit = \sqrt{\frac{6}{(fan\_in + fan\_out)}}$

4) He uniform → [-limit, limit]
↳ random weights

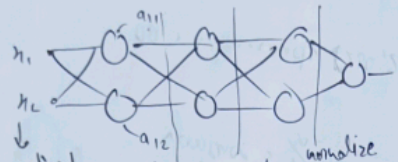
$limit = \sqrt{\frac{6}{(fan\_in)}}$

Some professors :-
$\sqrt{\frac{2}{fan\_in + fan\_out}}$

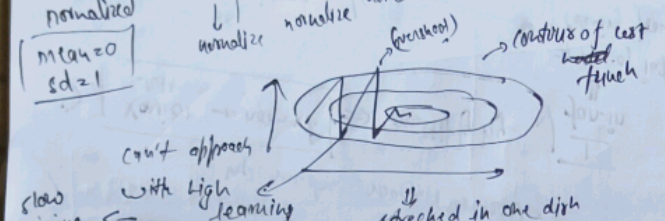
Batch normalization

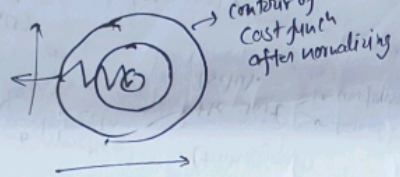
↳ makes training faster and stable

→ it consists of normalizing activation vectors from the hidden layers using the mean and variance of the current batch. This normalization step is applied right before (or right after) the nonlinear function.



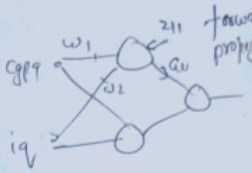
normalized: mean = 0, sd = 1





training is faster and stable.

cgpa	iq	placed
8.9	100	1
6.2	89	0
9.1	91	0
7.7	76	1
⋮	⋮	⋮
6.7	91	0



normalize: $\mu = 0$, $\sigma = 1$.

$z_{11} = w_1 \cdot \text{cgpa} + \text{weight}$

$g(z_{11}) = a_{11}$

Now there are two ways

$z_{11} \to z_{11}^N \to g(z_{11}) = a_{11}$

$z_{11} \to g(z_{11}) = a_{11} \to a_{11}^N$

$z_H^N = \frac{z_{11} - \mu}{\sigma}$ → standardization.

→ mini batches → layer by layer.

→ select batch size.

→ for this batch calculate μ, σ and further update.

say you took batch size of $= m$.

$(4,2) * (2,2) \to (4,2) + (1,2)$ (bias) $\to (4,2)$ (z)

(input weights)

stage 1

$\mu_B = \frac{1}{m} \sum_{i=1}^{m} z_{11}^i$

$\sigma_B = \sqrt{\frac{1}{m} \sum_{i=1}^{m} (z_{11}^i - \mu_B)}$ → two times separately for two nodes

$\left(z_{11}^{iH} = \frac{z_{11}^i - \mu_B}{\sigma + \varepsilon} \right)$ → error term if $\sigma = 0$, then no issue (0.00001).

stage 2

now

$z_{11}^{BN} = \gamma z_{11}^N + \beta$, γ, β → learnable parameter (changes with backpropagation)

initially $\gamma = \beta$, $\beta = 0$

⇓

different for each neuron

$g(z_{11}^{BN}) = a_{11}$.

* special case :-

Now suppose, $\gamma = \sigma + \varepsilon$, $\beta = \mu$

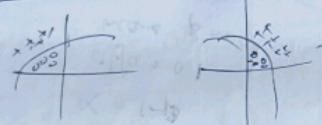
then $z_{11} \to z_{11}^N \to z_{11}^{BN}$

same

⇒ hence it is good for NN as some nodes don't require batch normalization; then it would be flexible to choose how much extent of normalization the model wants.

| stage 1 → normalization | stage 2 → scaling |

* Covariate shift :- ← (how stable?)

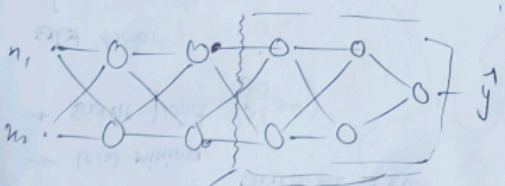


training data (model will not perform well) — test data (same covariance)

eg:- if you train your model for pictures of red rose, it won't perform well if test data contain different colour roses.

* internal covariate shift :-

↳ the change in the distribution of network activation due to change in network parameter during training



y

independent neural network.

⇓

the input that it is getting is constantly changing as the weights in the before NN also changing.

This training is unstable ← internal covariate shift

\# Exponentially weighted moving average

↳ won't train row by row

↳ training happen batch wise

1 batch

2 batch

last updated.

$\mu_1, \mu_{B2} \ldots \mu_{B15}$ → μ

Σ WMA

for every neuron → 4 parameters are stored after complete training

2 learnable → γ, β, 2 non-learnable → μ, σ (last updated)

* Advantage of batch norm

- stable
- faster (& 1)
- act as a regularizer
- reduce effect of weights

Optimizers in Deep Learning :-

challanges in GD :-
- to determine the learning rate
- learning rate scheduling.
- multidimension - many directions to move.
- local minima.
- Saddle point $\left(\frac{\partial L}{\partial w} = 0\right)$

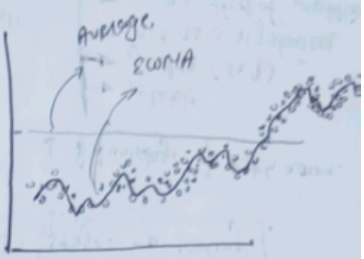
what to do :-
- prerequisite: Exponentially weighted moving Average
  - Momentum
  - Adagrad
  - NAG
  - RMSprop
  - Adam

Exponentially weighted Moving Average (EWMA) :-

↳ you find trends in time varying data.

used :-
- time series analysis
- financial
- signal process
- deep learning optimizers

the new point will have more importance or weightage is simple logic



$$V_t = \beta V_{t-1} + (1-\beta)\theta_t$$

$\beta \rightarrow$ constant : (0, 1)

index	temp(θ)
D_1	25
D_2	13
D_3	17
D_4	31
D_5	43

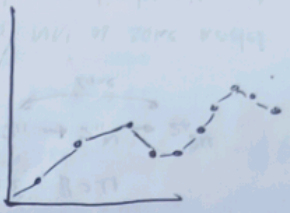
$V_0 = 0$ or $V_0 = \theta_0$. (say $R = 0.9$)

$V_1 = 0.9 \times V_0 + 0.1 \times 13$

$V_1 = 1.3$

$V_2 = 0.9 \times 1.3 + 0.1 \times 17$

$= 2.87$



what did this β signify?

if $\beta = 0.9$

it behaves as the no. we calculate with it is average of last $\frac{1}{1-\beta}$ days

i.e. $\frac{1}{1-0.9} = 10$ days.

in keras

$\alpha = 1 - \beta$

alfa = 0.1

means $\beta = 0.9$

if $V_0 = 0$

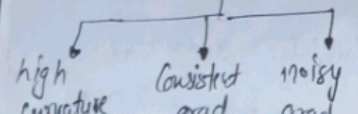
$$V_n = (1-\beta)\left[\beta^{n-1}\theta_1 + \beta^{n-2}\theta_2 + \dots + \beta\theta_{n-1} + \theta_n\right]$$

since $0 < \beta < 1$ $\quad \beta^a < \beta^b$ if $a < b$. ↓

thus the weightage of earlier value is less than newer.

Momentum Optimization :-

Non-Convex Optimization



Problem? → oscillates at bottom.

we do,

$w_{t+1} = w_t - \eta \nabla w_t$ → replace

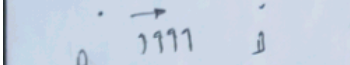
$v \rightarrow$ velocity

$w_{t+1} = w_t - v_t$

$v_t = \beta v_{t-1} + \eta \nabla w_t$

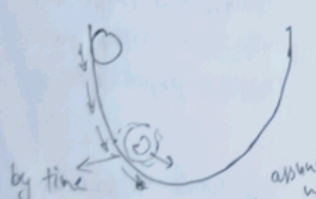
$0 < \beta < 1$

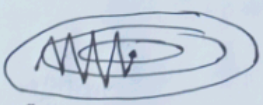
simple logic



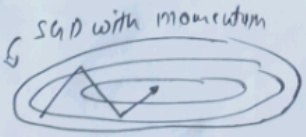
the more you gain info the more confident you are.

physics based intuition :-





normal SGD



SGD with momentum

$\beta \rightarrow$ hyperparameter

if $\beta = 0$ → normal SGD.

$\beta = 1$ → dynamic equilibrium (no decay)

NAG (Nesterov Accelerated Gradient) :- → uses damping to damp the oscillation.

in momentum optimization we do :-

history of velocity + gradient at that point ⇒ change.

in NAG :-

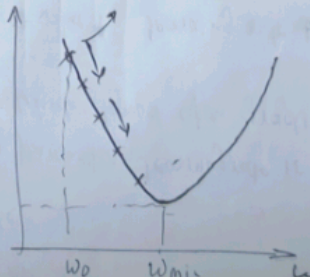
history of velocity ⇒ change, ⇒ gradient at that point (newer) ⇒ change

$w_{lq} = w_t - \beta V_{t-1}$

$V_t = \beta V_{t-1} + \eta \nabla w_{lq}$ → $V_t = (w_t - w_{lq}) + \eta \nabla w_{lq}$

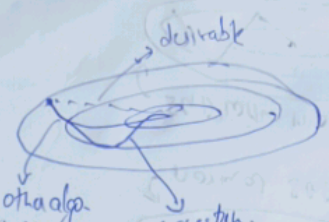
$w_{t+1} = w_t - V_t$

at every point you will calculate two things → gradient, momentum.

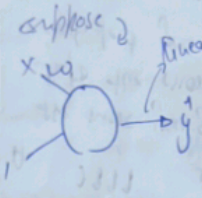


Adagrad optimization → what to use?
↳ adaptive gradient
↳ input feature scale different.
↳ features sparse (zero ↑)

in elongated contours it takes more time for other algorithms.



How adagrad work and fix this?
↳ different learning rate
⇒ simple logic ⇒ for every parameter learning rate is different which depend upon gradient value.
gradient value ↑ learning rate ↓
gradient value ↓ learning rate ↑



$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w}$$
$$= -2(y-\hat{y})x$$

now if x is sparse.
$\frac{\partial L}{\partial w} = \sum -2(y-\hat{y})x$ → for many terms
$\frac{\eta}{\partial w}$ → small
however $\frac{\partial L}{\partial b}$ → large as $b \geq 1$ for all input initially

BGD → sparse.
for i in epochs:
$w = w - \eta \frac{\partial L}{\partial w}$
$b = b - \eta \frac{\partial L}{\partial b}$

$$w_{t+1} = w_t - \frac{\eta \nabla w_t}{\sqrt{v_t + \varepsilon}}$$
where; $v_t = v_{t-1} + (\nabla w_t)^2$
extra terms
ε is to make sure non-zero denominator

that's why it is used in linear model only (not complex ones)
why sq? → we need to consider magnitude not direction of gradient

only directional priority.
↓↓
that's why not straight path

Disadvantage?
→ it can't converge to accurate answer
↓↓
as the I goes toward min, the learning rate which depends upon past gradient become too small
→ can't reach global minima

RMSprop (Root Mean Square prop):-
↳ improved version of adagrad.

formula:
$$v_t = \beta v_{t-1} + (1-\beta)(\nabla w_t)^2$$
$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \varepsilon}} \nabla w_t$$

β (0.95)

second advantage
Adagrad ×
RMSprop ✓
↓↓
global minima reach without getting stuck to local minima

$v_0 = 0$
$v_1 = 0.95 \times 0 + 0.05(\nabla w_1)^2$
$v_2 = 0.95 \times 0.05(\nabla w_1)^2 + 0.05(\nabla w_2)^2$
$v_3 = 0.95 \times 0.95 \times 0.05(\nabla w_1)^2 + 0.95 \times 0.05(\nabla w_2)^2 + 0.05(\nabla w_3)^2$
epoch 1 | epoch 2 | epoch 3

Verdict? → most cases → Adam then go to RMSprop
↓
hyperparameter tuning

Adam optimizer → it takes some information from other optimization technique
most powerful ← ANN, CNN, RNN
1) momentum
2) learning rate
↓
till now we are using these technique
↓
Adam merged these two.

formula for Adam:
(m_t → momentum term
v_t → learning rate optimization)

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{v_t + \varepsilon}} \cdot m_t$$

$m_t = \beta_1 m_{t-1} + (1-\beta_1) \nabla w_t$ → what we do in momentum
$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla w_t)^2$ → what we do in RMSprop and adagrad

Bias correction in
$$\hat{m_t} = \frac{m_t}{1-\beta_1^T} \qquad \hat{v_t} = \frac{v_t}{1-\beta_2^T}$$
T → epoch no.

why? → because $m_t = 0$ $v_t = 0$ initially
↓↓
to fix it we do this.

$\beta_1 \approx 0.9$
$\beta_2 \approx 0.99$
$\eta = 0.1, 0.01$
↓
automatically handled

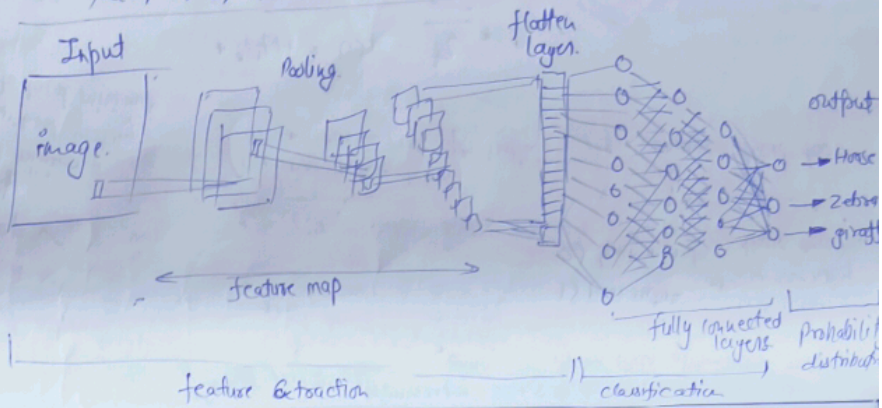
What is CNN ? → convolutional neural networks, also known as convnet, or CNNs

- are a special kind of neural networks for processing data that has a known grid-like topology like time-series (1D) or image (2D)
- has a "convolution layer" ⇒ perform convolution (⊛) ⇒ they help to extract primitive features of the data, at different level.
  - for eg:- first you detect edges, then shapes, then from then, you identify ears, nose, hair and from that you will classify face, arms, humans
- has 3 layers:-
  - convolutional layer
  - pooling layer
  - connected layer (ANN)
- inspired by cortex
- also we do various techniques
  - object localization → draw box around it then check it pixels and match them
  - object detection → gives probability also.
  - face detection
  - image segmentation
  - image resolution
  - colour images
  - pose detect.

Why not use ANN??

- we used ANN on MNIST data ⇓ 0.97 accu

1. High computation cost
2. overfitting
3. lose of impinto like spatial arrangement of pixel.



Basic of Images

- greyscale.
- RGB (coloured).

image → grid

→ each box has a colour between 0 - 255 ⇓ (0 - 1) normalize [for grey scale].

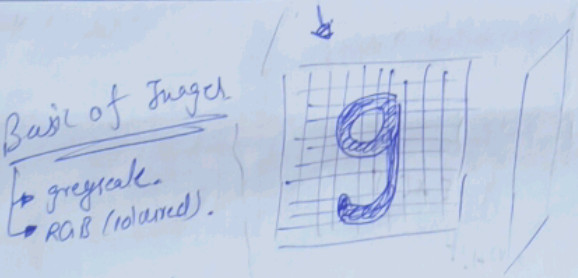
(2D numpy array)

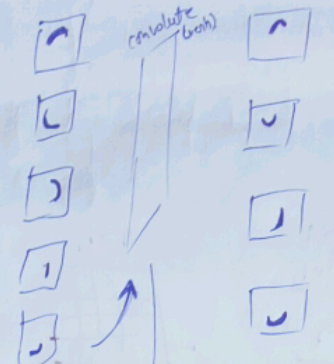
for coloured image ⇒ rgb code (primary colour red, green, blue) ⇓ each have 0 - 255 range mixed to form different colours

(rgb) 3, 28, 28, 8

(coloured image 3D numpy array)

Red → positive activation
blue → negative activation.

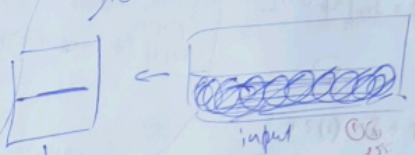


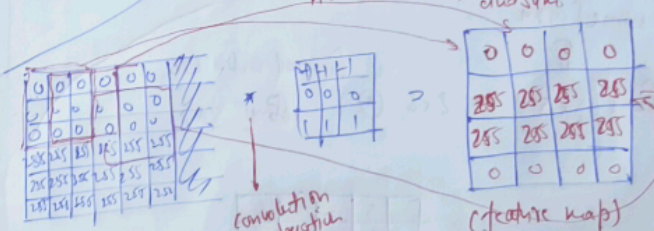


convolution → filter.

Edge detection:-

lets say you have this image





has to be 765, but since largest no can be 255 so scale it

vertical edge detector ⇒

-1	0	1
-1	0	1
-1	0	1

These filter values are not predefined, but it is made during back-propagation. Hence these filters act as weights in ANN